

IA Aplicada com LLMs

Aula 06 — Chunking e a medida da busca

Onde paramos

O vetor transforma texto em número, e o cosseno ordena. Duas perguntas ficaram abertas:

1. **Qual é a unidade** sobre a qual ele mede?
2. **Como se sabe** se a busca está boa?

A primeira é *chunking*. A segunda é a primeira medição de qualidade do curso — e o primeiro **dataset de avaliação**, que a Aula 11 vai retomar sob esse nome.

Parte 1

O chunk é a unidade de recuperação

O chunk é o que volta

O índice não devolve documentos. Devolve **chunks** — e o chunk chega ao leitor, humano ou modelo, **sem o texto em volta**.

Daí o critério:

Um chunk precisa fazer sentido isoladamente.

Um parágrafo que diga "*§2º Em viagem internacional, o limite de que trata o §1º passa a R\$ 260,00*" é inútil sozinho: não diz limite de quê.

Três estratégias, sobre o mesmo regulamento

Estratégia	Como corta	Custo
por caracteres	a cada N caracteres	parte regras ao meio
com sobreposição	idem, repetindo o fim do anterior	duplica conteúdo
por estrutura	artigo, parágrafo, inciso	exige que o documento tenha estrutura

Em documento normativo, **estrutura ganha** — e o aluno **mede isso**, em vez de aceitar a afirmação.

O documento já vem cortado pelo autor. Ignorar esse corte é jogar informação fora.

O mesmo artigo, cortado de três formas

[caracteres] "...limitado a R\$ 120,00 por pessoa por refeição, exigida a apresentação de nota fiscal. §2º Em viagem internacional, o limite de que trata o §1º passa a R\$ 260,00 por pes"
^ corta aqui

[estrutura] "Art. 4º §2º – Em viagem internacional, o limite de que trata o §1º passa a R\$ 260,00 por pessoa por refeição."
(+ cabeçalho do artigo herdado)

O primeiro perde o fim da regra. O segundo tem **exatamente uma regra**, com o contexto necessário para interpretá-la.

O tamanho, e o trade-off

chunk GRANDE	→	recupera contexto	→	desperdiça janela
chunk PEQUENO	→	é preciso	→	chega mutilado

Não há valor universal. Há uma **medição**, e ela é o Bloco 5.

E a decisão tem consequência na próxima aula: cada chunk recuperado ocupa a janela **em toda pergunta**.

A assimetria pergunta x documento

pergunta:	"posso reembolsar almoço em viagem?"	12 tokens
documento:	"Art. 4º §1º – O reembolso de refeições em viagem a serviço fica limitado a..."	64 tokens

Uma pergunta curta e um artigo longo **não têm a mesma forma**, e isso limita o cosseno mesmo quando o assunto é o mesmo.

O problema tem nome e tem soluções — e elas são **assunto da Aula 07**.

A outra assimetria: construir × consultar

A de cima é de **forma**. Esta é de **frequência**, e é a que justifica o índice:

construir o índice ...	1 chamada, com os 28 chunks juntos	UMA VEZ
esta consulta	1 chamada, só com a pergunta	A CADA PERGUNTA

Indexar é trabalho de **construção**; consultar é trabalho de **operação**.

Um sistema que reconstruísse o índice a cada pergunta não teria vantagem nenhuma sobre ler o documento inteiro.

Aqui o "uma vez" vale dentro de uma execução — o índice morre com o processo.

Na Aula 07 ele vai para disco. · Em **dinheiro**, Aula 13.

Parte 2

Medir a busca

O conjunto de perguntas com resposta conhecida

Dez perguntas, e para cada uma **qual artigo a responde**.

```
CONJUNTO = [  
    {"pergunta": "posso reembolsar almoço em viagem?", "artigo": "art-4-p1"},  
    {"pergunta": "e em viagem internacional?", "artigo": "art-4-p2"},  
    ...  
]
```

Construir isso é o trabalho. Medir é fácil depois.

Este é o primeiro *dataset de avaliação* do curso. A **Aula 11** o retoma sob esse nome, e a única diferença será a escala.

recall@k

$$\text{recall@}k = \frac{\#\{\text{perguntas cujo artigo certo est\'a entre os } k \text{ primeiros}\}}{\#\text{perguntas}}$$

```
def recall_at_k(conjunto, indice, k):  
    acertos = sum(caso["artigo"] in [c["artigo"] for c in  
                                   indice.buscar(caso["pergunta"], k)]  
                  for caso in conjunto)  
    return acertos / len(conjunto)
```

Mede a **busca**, e s\'o a busca. Se o artigo certo n\'ao vem, nenhuma gera\c3o\ salva a resposta.

Recall comparado

caracteres	9 chunks	recall@3 = 1/10 (10%)
sobreposto	12 chunks	recall@3 = 1/10 (10%)
estrutura	28 chunks	recall@3 = 10/10 (100%)



A diferença **não decorre do número de chunks** — a estratégia por estrutura produz mais chunks e menores.

Decorre de cada chunk conter **exatamente uma regra**, com o contexto necessário para interpretá-la.

A escolha do **k** é decisão de projeto

k alto	k baixo
esconde defeito de índice: um corte ruim é compensado por trazer mais coisa	expõe o defeito
enche a janela na Aula 07: cada chunk ocupa contexto em toda pergunta	contexto menor e mais preciso

```
k=1    recall = 7/10    ( 70%)
k=3    recall = 10/10   (100%)    <- estabiliza aqui
k=5    recall = 10/10   (100%)
k=10   recall = 10/10   (100%)
```

Reportar `recall@k` sem dizer o `k` é reportar nada. Aqui, `k=3` é o valor defensável: 5 e 10 pagam contexto por recall já ganho.

A pergunta que falha

Toda configuração tem uma. Achá-la e explicar por quê **vale mais que o recall médio.**

Diagnóstico	Sinal	Correção
Cegueira do vetor	depende de número, id, negação ou data	regra determinística
Defeito de corte	o trecho existe, mas partido ou sem cabeçalho	mudar a estratégia
Pergunta mal formulada	um humano com o documento também não saberia	corrigir o conjunto

Uma falha, diagnosticada

```
pergunta:  "quem aprova uma despesa de R$ 3.000?"  
esperado:  Art. 9º §2º      (de R$ 500,00 a R$ 5.000,00)  
obtido:    Art. 9º §1º      (até R$ 500,00)  
           Art. 9º §3º      (acima de R$ 5.000,00)
```

Os três parágrafos tratam de alçadas e se distinguem **apenas por faixas de valor**.

Diagnóstico: o vetor não representa magnitude.

A correção não é de chunking — é filtrar a faixa **em código**, antes de ordenar por similaridade.

O carimbo cresce

Aula	Campos
03	prompt · modelo · parâmetros
05	arquitetura
06	estratégia de <i>chunking</i> · modelo de embedding

Trocar o corte **invalida a comparação de** `recall` com as execuções anteriores — exatamente como trocar o prompt invalidava a suíte da Aula 03.

E reindexar com outro modelo de embedding muda o `recall` **sem que ninguém tenha mudado o código.**

O que fica

- O **chunk** é a unidade de recuperação — é ele que chega ao leitor
- Em documento normativo, **estrutura ganha** — e a medição prova
- Reportar `recall@k` sem dizer o `k` é **reportar nada**
- A pergunta que falha vale mais que a média

Próxima nota: o mesmo `ranquear`, com o corpus trocado por **exemplares de rota** — e a dívida que a Aula 05 deixou escrita em código.